



Откройте новый уровень в карьере



86.43

Общий рейтинг

Нетология

Знания, на которые можно положиться

[Подписаться](#)

kirakirap

21 мая в 14:14

Гайд по Scikit-learn в 2025: собираем пайплайн, который не сломается



Простой



30 мин



10К

Блог компании Нетология, Машинное обучение*, Python*, Data Engineering*, Data Mining*

[Тutorial](#)

Scikit-learn — это одна из основных Python-библиотек для машинного обучения. Её подключают в прикладных проектах, AutoML-системах и учебных курсах — как базовый инструмент для работы с моделями. Даже если вы давно пишете с PyTorch или CatBoost, в задачах с табличными данными, скорее всего, всё ещё вызываете `fit`, `predict`, `score` — через `sklearn`.

В 2025 году в библиотеку добавили несколько важных обновлений: доработали работу с пайплайнами, подключили полную поддержку `pandas API`, упростили контроль за экспериментами.

Мы подготовили гайд, как работать со `scikit-learn` в 2025 году. Новичкам он поможет собрать первую ML-задачу. А тем, кто уже использует библиотеку, — освежить знания и понять, что изменилось в новых версиях.

**Александр Немальцев**Руководитель группы машинного обучения в Сбере,
консультировал при подготовке гайда

Для навигации по гайду:

[Почему scikit-learn по-прежнему актуальна](#)[Как установить sklearn и подготовить окружение](#)[Как устроена библиотека scikit-learn и с чего начать работу](#)[Подготовка данных к обучению](#)[Классификация и регрессия в sklearn](#)[Подбор параметров и валидация](#)[Разбираем частые ошибки при работе с библиотекой](#)[Что изменилось в scikit-learn в 2025 году](#)

РЕКЛАМА

**Ты точно знаешь себя?**

Проверь свой IT-профиль



Почему scikit-learn по-прежнему актуальна

Sklearn остаётся в стеке, потому что хорошо закрывает типовые задачи ML: классификацию, регрессию, отбор признаков, масштабирование, кросс-валидацию. Когда данные уже лежат в таблице, а нужно быстро запустить задачу, с ней проще всего собрать работающий прототип.

Модели и трансформеры в scikit-learn реализуют единый набор основных методов. Этот подход давно подхватили другие библиотеки, например LightGBM и Optuna. За счёт этого инструменты хорошо сочетаются, а модели можно заменять без полного изменения кода.

Сильная сторона sklearn — пайплайны. Их используют, чтобы собрать обработку и обучение в одну цепочку. Вместо того чтобы по отдельности вызывать масштабирование, кодировать признаки и передавать всё в модель, шаги объединяются через Pipeline или ColumnTransformer — если к разным признакам, например числовым и категориальным, нужно применять разные преобразования. Это сохраняет порядок, убирает лишнюю ручную логику и снижает количество багов.

Библиотека sklearn хорошо сочетается с остальными инструментами Python. Она работает с массивами numpy и датафреймами pandas, строит графики через matplotlib, сохраняет модели с помощью joblib. В пайплайн можно включать и сторонние модели — если у них такой же интерфейс. Например, CatBoostClassifier с методами fit и predict спокойно встраивается без дополнительной обёртки.

Библиотеку используют и в обучении, и в прикладной разработке. На курсах scikit-learn дают как первый инструмент: на нём проще всего показать, как устроена ML-задача.



В рабочих проектах scikit-learn остаётся в пайплайне — для предобработки, оценки метрик или как базовая модель. А если приоритет — читаемость, стабильность и совместимость со стеком, её используют и в финальной сборке.

Теперь разберёмся, как работать с библиотекой в 2025 году.

Как установить sklearn и подготовить окружение



Ты точно знаешь себя?

Проверь свой IT-профиль

Освоить ключевой инструмент для анализа данных и машинного обучения, а также научиться работать с данными на всех этапах — от сбора, очистки, анализа до визуализации — можно за 4 месяца на курсе «Python для анализа данных». Обучаем по государственной лицензии и выдаём удостоверение о повышении квалификации установленного образца, которое можно показать работодателю.

Ниже — полный порядок настройки, но если вы хотите быстро попробовать scikit-learn, достаточно первого шага. После установки можно сразу запускать код, импортировать модели и строить простые пайплайны.

Шаг 1. Установите библиотеку через pip

Установка sklearn выполняется одной командой через консоль:

```
pip install scikit-learn
```

Объяснить код с 

При установке автоматически подтягиваются библиотеки, которые нужны для работы: numpy , scipy , joblib , threadpoolctl .

Библиотека scikit-learn поддерживает Python 3.7–3.12. Если версия ниже, она может не установиться или не запуститься. Проверить версию можно так:

```
python --version
```

Объяснить код с 

Если на компьютере несколько версий Python, pip может ссылаться не на ту. Чтобы избежать этого, используйте явный вызов:

```
python -m pip install scikit-learn
```

Объяснить код с 

Шаг 2. Создайте виртуальное окружение

Чтобы пакеты из одного проекта не мешали другому, лучше создать отдельное окружение. Иначе может оказаться, что один проект требует, например, pandas 2.2, а другой — только 1.5. Если всё устанавливать в систему без изоляции, версии начнут конфликтовать и часть кода просто не запустится.

Создать окружение можно так:

```
python -m venv venv
source venv/bin/activate # Для Linux/macOS
```



Ты точно знаешь себя?

Проверь свой IT-профиль

Вместо стандартного `venv` можно использовать `poetry` или `conda`. Эти инструменты делают то же самое, но дополнительно помогают фиксировать версии библиотек и переключаться между проектами. Они автоматически привязывают окружение к конкретной папке проекта, так что достаточно перейти в нужную директорию и активировать среду командой `poetry shell` или `conda activate myenv`.

Оба инструмента нужно установить отдельно:

- `poetry` — через `curl` или `pip`;
- `conda` — через установщик `Anaconda` или `Miniconda`.

Первый подойдёт, если нужно сразу готовое окружение с Jupyter и NumPy, второй — если хотите настроить всё с нуля и сэкономить место.

Пример с poetry:

```
poetry new my_project
cd my_project
poetry add scikit-learn
```

Объяснить код с 

Пример с conda:

```
conda create -n myenv python=3.11 scikit-learn
conda activate myenv
```

Объяснить код с 

Команды подойдут для macOS, Linux и Windows: в терминале они работают одинаково.

Шаг 3. Установите сопутствующие библиотеки

Эти пакеты не входят в `scikit-learn`, но почти всегда используются вместе с ней:

- `pandas` — для работы с табличными данными;
- `matplotlib`, `seaborn` — для визуализации.

Команда для установки:

```
pip install pandas matplotlib seaborn
```

Объяснить код с 

Если в окружении уже были установлены другие версии библиотек, `pip` может выдать ошибку



Ты точно знаешь себя?

Проверь свой IT-профиль

`pip check`

[Объяснить код с !\[\]\(2e897e890e69d81eae4503a8342c36b0_img.jpg\)](#)

Если появляются конфликты, проще пересоздать окружение и установить всё заново.

Шаг 4. Выберите среду для запуска

Библиотека `sklearn` работает в любом редакторе: Jupyter, VS Code, PyCharm — или просто в терминале. На старте чаще всего используют Jupyter Notebook: в нём удобно экспериментировать с кодом — запускать ячейки, менять параметры и сразу видеть, как ведут себя данные.

Чтобы запустить Jupyter Notebook, используйте команду:

```
pip install notebook
jupyter notebook
```

[Объяснить код с !\[\]\(0b5e7e25e8775f7e7e80906ada4f0021_img.jpg\)](#)

Откроется вкладка в браузере. Нажмите **New** → **Python 3**, чтобы создать новый ноутбук.

В первой ячейке проверьте, что библиотека установилась:

```
import sklearn
print(sklearn.__version__)
```

[Объяснить код с !\[\]\(bd3b31712ad9bab5a241210fa6925cdd_img.jpg\)](#)

Если ошибок нет и версия отображается — установка прошла успешно.

Дальше можно убедиться, что всё работает корректно: библиотека загружается, данные читаются, модель обучается. Для этого запустите простой пример на встроенном датасете `iris`:

```
from sklearn.datasets import load_iris
from sklearn.linear_model import LogisticRegression

X, y = load_iris(return_X_y=True)
model = LogisticRegression().fit(X, y)
```

[Объяснить код с !\[\]\(7bc43b319a082987e20f7bf78f4bab80_img.jpg\)](#)

Если код выполняется без ошибок — `sklearn` готова к работе.

[К содержанию](#)

Как устроена библиотека `scikit-learn` и с чего начать работу



Ты точно знаешь себя?

Проверь свой IT-профиль

из `metrics`. Эта схема работает одинаково для разных алгоритмов, поэтому после первого примера становится понятно, как работает вся библиотека.

На старте обычно подключают четыре модуля:

- `datasets` — встроенные датасеты для тестов;
- `linear_model` — простые модели вроде логистической регрессии;
- `model_selection` — разбиение данных и кросс-валидация;
- `metrics` — функции для оценки качества.

Пример: простая задача классификации на встроенном датасете `iris` :

```
from sklearn import datasets, linear_model, metrics, model_selection

# Загружаем данные
X, y = datasets.load_iris(return_X_y=True)

# Разбиваем на обучение и тест
X_train, X_test, y_train, y_test = model_selection.train_test_split(X, y, test_size=0.3, random_state=42)

# Обучаем модель
model = linear_model.LogisticRegression(max_iter=1000)
model.fit(X_train, y_train)

# Получаем предсказания
y_pred = model.predict(X_test)

# Считаем метрику
accuracy = metrics.accuracy_score(y_test, y_pred)
print(f"Accuracy: {accuracy:.2f}")
```

Объяснить код с 

Здесь загружается встроенный набор данных `iris`, где каждый объект — это параметры цветка, а цель — определить его класс. Данные делятся на обучающую и тестовую выборки: модель учится на одной части (`fit`) и проверяется на другой (`predict`). После этого считается метрика качества — точность (`accuracy`), то есть доля правильных ответов.

На этом этапе важно понимать, как именно делятся данные. По умолчанию `train_test_split` просто случайно выбирает, какие объекты пойдут в обучение, а какие — в тест. Но поведение этой функции можно настраивать:

- `test_size` — доля тестовой выборки. По умолчанию 0.25, можно задать любое значение от 0 до 1;
- `train_size` — доля обучающей выборки. Если указано одновременно с `test_size`, они должны суммироваться до 1;
- `random_state` — фиксирует результат, чтобы при каждом запуске разбиение было одинаковым. Число может быть любым — главное, чтобы оно не менялось между запусками. В примерах часто используют 42 просто по традиции.
- `stratify=y` — сохраняет соотношение классов в целевой переменной, как было в исходных данных. Это важно, если один из классов сильно преобладает — например, 90% объектов



Ты точно знаешь себя?

Проверь свой IT-профиль

можно передавать сразу несколько массивов — например: `X`, `y` и веса.

В нашем коде используется `test_size=0.3`, чтобы 30% данных попали в тест, и `random_state=42` для воспроизводимости. `stratify` помогает сохранить соотношение классов.

Мы разобрали минимальный рабочий пример, как устроена задача машинного обучения: есть данные, есть модель, есть оценка. Всё остальное: отбор признаков, обработка, выбор алгоритма — добавляется к этой базовой схеме.

В нашем случае данные передаются без предварительной обработки, но на практике она нужна почти всегда. Разберём, как подготовить данные для использования в `sklearn`.

К содержанию

Подготовка данных к обучению

Scikit-learn принимает числовые и категориальные данные, но требует предварительной подготовки. Обычно в таблицах бывают пропуски, текстовые значения и признаки с разными шкалами — их нужно обработать заранее.

В новых версиях `scikit-learn` некоторые модели, например `HistGradientBoostingClassifier`, умеют работать с пропусками, тем не менее в большинстве случаев NaN нужно обрабатывать отдельно — до запуска обучения.



Источник

Сначала смотрят структуру, потом заполняют пропуски, кодируют категории и масштабируют значения — если это влияет на модель. Эти шаги можно собрать в один пайплайн, но сначала разберём их по отдельности.

Проверяем структуру и типы данных

Первый шаг — понять, как устроена таблица. Какие признаки числовые, какие — категориальные, где есть пропуски. Это определяет, какие трансформеры придётся подключить и в каком порядке выстраивать обработку.



Ты точно знаешь себя?

Проверь свой IT-профиль

```
import pandas as pd
```

```
df = pd.read_csv("your_data.csv")
print(df.info())
print(df.head())
```

[Объяснить код с !\[\]\(3d8c13c92b853674f749aac6fa869926_img.jpg\)](#)

`info()` показывает типы колонок и пропуски, `head()` — пример содержимого. Этого достаточно, чтобы понять, что требует обработки.

Если `pandas` уже подключена, повторно импортировать её не нужно. В ноутбуке или скрипте всё работает в одном окружении, и одного импорта достаточно. Но если вы копируете код отдельно или начинаете новый файл, `import` стоит указать явно.

Обрабатываем пропуски

Scikit-learn не работает с `NaN`. Если в данных остались пропуски, `fit()` просто не запустится. Их нужно убрать до обучения — для этого используют `SimpleImputer`.

Пропуски можно заполнить разными способами:

- средним значением по колонке (`mean`);
- медианой (`median`);
- фиксированным значением (`constant`), например 0 или `"missing"`.

Сначала создаём объект с нужным способом заполнения: `mean`, `median` или `constant`. Потом вызываем `fit()` на обучающей выборке — он считает, чем заменять. Дальше применяем `transform()` отдельно к `train` и `test`. Это важно, чтобы не утекла информация с теста в обучение.

Пример запроса:

```
from sklearn.impute import SimpleImputer

imputer = SimpleImputer(strategy="mean")
imputer.fit(X_train)
X_train_filled = imputer.transform(X_train)
X_test_filled = imputer.transform(X_test)
```

[Объяснить код с !\[\]\(e9474ce1d70442456f8fe9c393ea149c_img.jpg\)](#)

Приводим числовые признаки к одной шкале

Некоторые модели чувствительны к диапазону значений — например, те, что используют расстояния между объектами (как `k`-ближайших соседей) или градиенты (линейные модели, нейросети). Если один признак меняется от 0 до 1, а другой — от 0 до 10 000, вес второго окажется сильно завышен.

Чтобы этого избежать, признаки приводят к одной шкале. В `scikit-learn` для этого есть два основных инструмента:



Ты точно знаешь себя?

Проверь свой IT-профиль

Масштабирование делают так же, как и заполнение пропусков: `fit()` — на `X_train`, `transform()` — на обе выборки.

Пример запроса:

```
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()
scaler.fit(X_train)
X_train_scaled = scaler.transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

[Объяснить код с !\[\]\(71ceb62b681518c82e95d615e7265d66_img.jpg\)](#)

Если модель построена на деревьях решений — например, случайный лес или градиентный бустинг, — масштабировать признаки не нужно: такие алгоритмы не зависят от шкалы.

Кодируем категориальные признаки

Модели scikit-learn работают только с числами. Если в таблице есть текст — например, "красный", "синий", "зелёный" в колонке "цвет", — его нужно превратить в числовые значения. Иначе `fit()` просто не запустится.

Для этого используют `OneHotEncoder`. Он создаёт отдельный признак для каждого уникального значения. Вместо одной колонки "цвет" получится несколько: "цвет_красный", "цвет_синий", "цвет_зелёный" и т. д.

По умолчанию `OneHotEncoder` возвращает разреженную матрицу — в ней не хранятся нули. Чтобы получить обычный `numpy`-массив, указывают `sparse_output=False`.

Пример запроса:

```
from sklearn.preprocessing import OneHotEncoder

encoder = OneHotEncoder(sparse_output=False, handle_unknown="ignore")
encoder.fit(X_train[["цвет"]])
X_train_encoded = encoder.transform(X_train[["цвет"]])
X_test_encoded = encoder.transform(X_test[["цвет"]])
```

[Объяснить код с !\[\]\(d5831b2ac75eb48b4c49d27e61d24c03_img.jpg\)](#)

Параметр `handle_unknown="ignore"` нужен, чтобы не получить ошибку, если в тесте появится значение, которого не было в обучении.

Если категорий много или они упорядочены по смыслу, можно использовать `OrdinalEncoder`. Но для начала чаще выбирают one-hot. Он проще: не нужно задавать порядок категорий — и безопаснее: модель не подхватит лишние зависимости между значениями, как это бывает с `OrdinalEncoder`.

Назначаем трансформеры по группам признаков

Числовые и категориальные данные обрабатываются по-разному. One-hot не применим к числам, а масштабирование — к строкам. Поэтому каждому типу назначают свой трансформер — объект,

**Ты точно знаешь себя?**

Проверь свой IT-профиль

столбцам применять, и собирает всё в одну структуру. Например, "age" и "income" масштабируются, "city" — кодируется.

Важно: строковые признаки должны быть приведены к типу `category` — иначе `make_column_selector` с `dtype_include="category"` их не увидит. По умолчанию они обычно имеют тип `object`.

Пример кода:

```
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import StandardScaler, OneHotEncoder

preprocessor = ColumnTransformer([
    ("num", StandardScaler(), make_column_selector(dtype_include="number")),
    ("cat", OneHotEncoder(), make_column_selector(dtype_include="category")),
])
```

Объяснить код с 

Теперь `fit()` и `transform()` можно вызывать сразу на всей таблице: каждая колонка пройдёт свою обработку.

Автоматический выбор колонок работает по типам: числовые, категориальные. Обычно этого хватает, но если типы данных определились неправильно, нужные признаки могут не попасть в обработку. В таких случаях колонки лучше задать через `feature_names_in_` и `feature_types_in_`. Оба варианта упрощают построение пайплайна: не нужно вручную фильтровать признаки под каждый трансформер.

Собираем обработку и модель в один пайплайн

Если подготовка включает несколько шагов: заполнение пропусков, масштабирование, кодирование, — её удобнее оформить как цепочку через `Pipeline`. Это убирает лишние вызовы `fit()` и `transform()`, упрощает код и избавляет от повторов в функциональной логике.

```
from sklearn.pipeline import Pipeline
from sklearn.linear_model import LogisticRegression

model = Pipeline(steps=[
    ("preprocess", preprocessor),
    ("classifier", LogisticRegression(max_iter=1000))
])
```

Объяснить код с 

Теперь достаточно вызвать `fit()` один раз — `scikit-learn` сам применит все шаги в нужном порядке: сначала обработку, потом обучение. Каждый шаг (`step`) — это отдельный трансформер или модель внутри пайплайна.

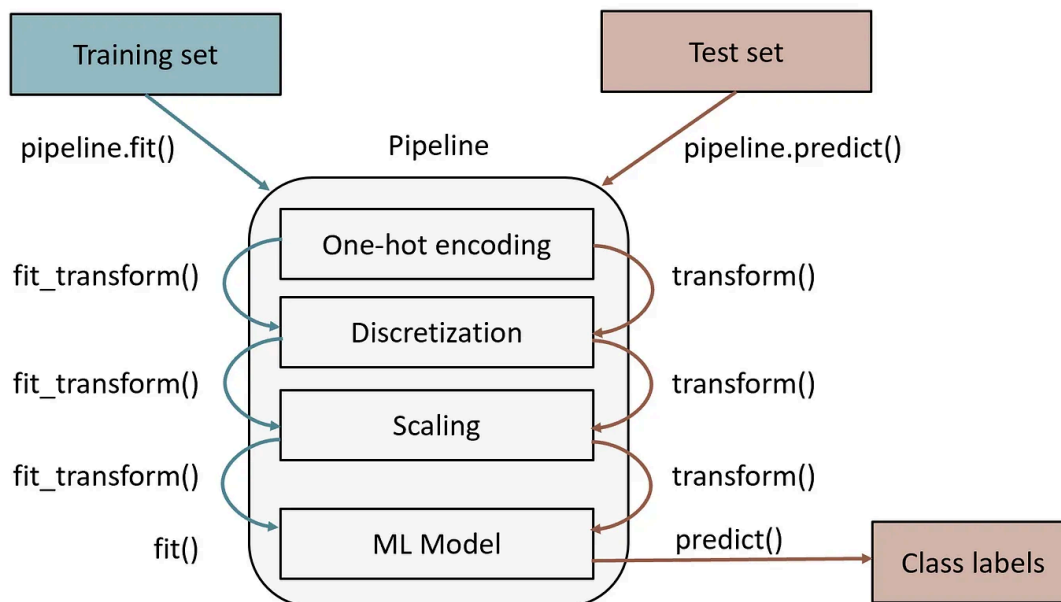
На `predict()` сработает та же цепочка, но без повторного `fit()`.

Важно: `fit()` вызывается только на обучающей выборке — он считает параметры, например среднее для масштабирования. `transform()` применяет эти параметры к `train` и `test`. Если случайно вызвать `fit()` на тесте, модель увидит данные, которых не должно быть, и



Ты точно знаешь себя?

Проверь свой IT-профиль



Пайплайн в библиотеке scikit-learn. Источник

[К содержанию](#)

Классификация и регрессия в sklearn

Когда данные подготовлены и пайплайн собран, можно переходить к обучению моделей. Как мы уже разбирали, в scikit-learn у всех алгоритмов общий интерфейс: `fit()` обучает, `predict()` делает предсказания, `score()` оценивает результат.

Классификация и регрессия работают по одной схеме и различаются только типом целевой переменной: в первом случае это класс, во втором — число.

Рассказываем, как выбрать модель под задачу, обучить её и оценить качество предсказаний.

Выбираем модель под задачу

Среди классификаторов sklearn чаще всего используют:

- `LogisticRegression` — подойдёт, если данные простые и хочется понять, как работает модель;
- `RandomForestClassifier` — хорошо справляется с шумом и разными типами признаков;
- `GradientBoostingClassifier` — даёт точные результаты, если задача сложная и данных недостаточно для обучения.

Из регрессионных моделей:

- `LinearRegression` — если признаки числовые и зависимость от целевой переменной простая;
- `RandomForestRegressor` — устойчив к шуму и выбросам, подходит для разнородных данных;



Ты точно знаешь себя?
Проверь свой IT-профиль

Подробные описания есть в документации библиотеки scikit-learn.

Если задача новая, обычно сравнивают простую и более сложную модель с настройками по умолчанию. Если качество заметно отличается, сначала можно попробовать доработать простую модель, а при необходимости — перейти к более сложной. Иногда разница бывает настолько большой, что имеет смысл сразу выбрать более сложный алгоритм.

Обучаем модель на данных

Когда модель выбрана, запускаем обучение через метод `fit()`. Он принимает обучающую выборку: признаки (`X_train`) и правильные ответы (`y_train`).

```
model.fit(X_train, y_train)
```

Объяснить код с 

Если используется `Pipeline`, внутри `fit()` последовательно выполняются все шаги обработки: сначала, например, заполняются пропуски и масштабируются признаки, потом запускается обучение модели. Никаких дополнительных вызовов не нужно.

Вот как это выглядит:

```
pipeline.fit(X_train, y_train)      # Обучает и обрабатывает данные
y_pred = pipeline.predict(X_test)   # Делает предсказания на тестовой выборке
```

Объяснить код с 

Для проверки можно использовать встроенные датасеты — например, `load_iris()` для классификации или `load_diabetes()` для регрессии. Это простой способ убедиться, что пайплайн собран правильно и всё запускается без ошибок.

Оцениваем качество модели

После обучения нужно понять, как модель справляется с задачей. У большинства моделей в `scikit-learn` есть метод `score()`, который по умолчанию возвращает одну базовую метрику:

- для классификации — `accuracy` (доля правильных ответов);
- для регрессии — R^2 (насколько модель объясняет разброс в данных).

Для быстрой проверки этого достаточно, но чаще используют более точные метрики — когда классы несбалансированны или важно понять, насколько сильно результат отклоняется от правильного значения.

Для классификации:

- `accuracy_score` — это та же метрика, которую по умолчанию возвращает `score()`, но её часто вызывают отдельно, чтобы сравнить несколько моделей или сохранить результат в общей таблице.

Подходит, если классы в выборке распределены равномерно. Если один класс встречается



Ты точно знаешь себя?

Проверь свой IT-профиль

- `f1_score` — полезен, когда важно учитывать оба типа ошибок: и ложные срабатывания, и пропущенные случаи. Это усреднённый показатель между `precision` (точность) и `recall` (полнота). Его используют, если данные несбалансированны или нужно ловить редкие случаи.

- `roc_auc_score` — показывает, насколько хорошо модель различает классы по предсказанным вероятностям. Часто используется в бинарной классификации. В многоклассовых задачах его тоже применяют — через подход `one-vs-rest`.

Для регрессии:

- `r2_score` — та же метрика, которую по умолчанию возвращает `score()` у регрессионных моделей. Её часто вызывают явно — чтобы сравнить с другими метриками, сохранить результат или посчитать на кросс-валидации.

Показывает, насколько хорошо модель объясняет разброс в целевой переменной. Может принимать значения от 1 (идеально) до отрицательных (хуже, чем просто усреднение).

- `mean_absolute_error` (MAE) — показывает, насколько в среднем предсказания отклоняются от фактических значений. Все ошибки учитываются одинаково, независимо от их величины.
- `mean_squared_error` (MSE) — тоже измеряет отклонения, но в квадрате. За счёт этого большие ошибки влияют на результат сильнее. Если важно учитывать именно большие отклонения — лучше смотреть на MSE.

Все метрики вызываются из `sklearn.metrics`. Вот пример для классификации и регрессии:

```
from sklearn import metrics

# Метрики классификации
accuracy = metrics.accuracy_score(y_test, y_pred)      # Доля правильных ответов
f1 = metrics.f1_score(y_test, y_pred)                  # Баланс между точностью и полнотой
roc_auc = metrics.roc_auc_score(y_test, model.predict_proba(X_test)[: , 1])
# Насколько хорошо модель разделяет классы по вероятностям (для бинарной классификации)

# Метрики регрессии
mae = metrics.mean_absolute_error(y_test, y_pred)      # Среднее абсолютное отклонение
mse = metrics.mean_squared_error(y_test, y_pred)       # Среднеквадратичное отклонение
r2 = metrics.r2_score(y_test, y_pred)                  # Насколько модель объясняет разброс в
```

Объяснить код с 

Сравниваем несколько моделей

Чтобы понять, какая модель лучше справляется с задачей, обычно тестируют несколько — на одних и тех же данных и с одинаковой обработкой. Это даёт честное сравнение: все алгоритмы получают одинаковый вход и работают в равных условиях.

Для оценки используют не один запуск, а кросс-валидацию или повторные разбиения. Так видно, насколько стабильно модель держит результат. Если метрики сильно меняются, модель может быть чувствительной к выборке.

Результаты удобно сводить в таблицу: `accuracy`, `f1`, `roc_auc` — для классификации; `MAE`, `MSE`, `R2` — для регрессии. Это помогает понять, где разница в качестве действительно есть, а где модели работают на одном уровне.

Обычно это выглядит так: задаётся общая обработка данных, по очереди подключаются разные



Ты точно знаешь себя?

Проверь свой IT-профиль

Финальный набор для стартовой задачи

Первая рабочая задача в sklearn всегда строится из одних и тех же компонентов: данные, обработка, модель и метрика. Всё это можно собрать в один эксперимент. Он показывает, как устроена задача, помогает проверить устойчивость модели и даёт основу для следующего шага: настройки, сравнения или выхода в прод.

Вот пример: классификация с логистической регрессией и бустингом на одних и тех же данных. Обработка задаётся один раз, модели переключаются в цикле, результат оценивается по accuracy и f1.

```
from sklearn import datasets, model_selection, preprocessing, pipeline, linear_model, ensemble

# Данные
X, y = datasets.load_iris(return_X_y=True)
X_train, X_test, y_train, y_test = model_selection.train_test_split(X, y, random_state=42)

# Обработка
scaler = preprocessing.StandardScaler()

# Модели
models = {
    "LogisticRegression": linear_model.LogisticRegression(max_iter=1000),
    "GradientBoosting": ensemble.GradientBoostingClassifier()
}

# Запускаем цикл
for name, model in models.items():
    clf = pipeline.make_pipeline(scaler, model)
    clf.fit(X_train, y_train)
    y_pred = clf.predict(X_test)
    acc = metrics.accuracy_score(y_test, y_pred)
    f1 = metrics.f1_score(y_test, y_pred, average='macro')
    print(f"{name}: accuracy = {acc:.2f}, f1 = {f1:.2f}")
```

Объяснить код с 

Этот шаблон легко адаптировать под задачу: заменить данные, добавить кодирование или обработку пропусков, переключить модели или метрики. Структура при этом остаётся той же.

[К содержанию](#)

Кросс-валидация и подбор параметров

После первого эксперимента обычно переходят к оценке и настройке модели. Нужно понять, насколько стабильный результат она даёт и можно ли его улучшить.

Сначала проверяют, как модель ведёт себя на разных разбиениях данных. Для этого используют кросс-валидацию. Она помогает понять, будет ли модель хорошо работать на новых данных, а не только на одной удачно выбранной выборке.

Параллельно начинают подбирать параметры: пробуют разные значения и смотрят, как они влияют на метрику. Если параметров немного, это можно сделать вручную. Когда вариантов становится



Ты точно знаешь себя?

Проверь свой IT-профиль

запускать перебор в scikit-learn.

Запускаем кросс-валидацию

Один запуск даёт общее представление о качестве, но результат может случайно оказаться лучше или хуже реального. Поэтому чаще используют кросс-валидацию, если только её применение не ограничено задачей, как, например, в потоковых данных или при разметке по времени.

Кросс-валидация проверяет модель на нескольких разбиениях. Данные делятся на фолды — обычно 5 или 10, — и модель по очереди обучается на одних фрагментах, тестируется на других. Так можно понять, насколько стабильно она ведёт себя в разных условиях.

Для запуска кросс-валидации в `scikit-learn` есть функция `cross_val_score`. Она возвращает оценки на каждом фолде, а дальше можно посчитать среднее или разброс значений.

Пример:

```
from sklearn.model_selection import cross_val_score
from sklearn.linear_model import LogisticRegression

model = LogisticRegression()

# Проверяем точность модели на 5 фолдах
scores = cross_val_score(model, X, y, cv=5, scoring="accuracy")

print(scores)
print(f"Средняя точность: {scores.mean():.2f}")
```

Объяснить код с 

Число фолдов задаётся через параметр `cv`. По умолчанию во всех вариантах кросс-валидации применяется `KFold` — равномерное деление без учёта классов или групп. Обычно используют 5 или 10 фолдов: это стандартные значения для `cv`.

Кросс-валидация работает и с отдельной моделью, и с пайплайном. Главное — чтобы все шаги обработки были внутри общей структуры. Тогда порядок сохраняется и модель тестируется на данных, которые не использовались в обучении. Иначе оценки могут получиться завышенными — просто потому что информация утекла из теста.

Подбираем параметры

В большинстве алгоритмов `scikit-learn` есть настройки, от которых зависит точность, устойчивость и скорость. Чтобы начать их подбирать, нужно разобраться, какие параметры за что отвечают и на что влияют.

В логистической регрессии (`LogisticRegression`) это, например:

- `C` — контролирует величину штрафа за большие веса: чем меньше значение, тем жёстче ограничения;
- `penalty` — задаёт тип регуляризации: `l1`, `l2` или их комбинация — `elasticnet`;
- `solver` — определяет алгоритм оптимизации, который используется при обучении модели.

Для деревьев и ансамблей — `DecisionTreeClassifier`, `RandomForestClassifier`,



Ты точно знаешь себя?

Проверь свой IT-профиль

- `max_depth` — ограничивает глубину дерева. Чем меньше значение, тем проще структура;
- `n_estimators` — задаёт количество деревьев в ансамбле;

- `min_samples_split` — устанавливает минимальное число объектов, при котором узел может быть разделён;
- `learning_rate` — уменьшает вклад каждого нового дерева в итоговую модель — в бустинге.

Полный список параметров можно получить через `.get_params()` или в документации к нужной модели. Но на старте достаточно выбрать 2–3 главных — те, что действительно влияют на результат, — и попробовать разные значения.

Самый простой способ — задать параметры вручную. Например, изменить `C` у логистической регрессии или `max_depth` у дерева. Это поможет понять, на что реагирует модель и в каком диапазоне стоит продолжать подбор.

Если параметров становится больше или важно проверить их в разных сочетаниях, удобнее использовать `GridSearchCV`. Он перебирает заданные комбинации и для каждой запускает кросс-валидацию, чтобы оценить результат. Так можно найти устойчивое решение, а не просто совпадение на одной выборке.

Вот пример: подбор `C` и `penalty` для логистической регрессии с помощью `GridSearchCV`:

```
from sklearn.model_selection import GridSearchCV
from sklearn.linear_model import LogisticRegression

# Задаём параметры, которые хотим перебрать
params = {
    "C": [0.1, 1, 10],
    "penalty": ["l2"],
    "solver": ["liblinear"]
}

# Оборачиваем модель в GridSearchCV и указываем число разбиений
grid = GridSearchCV(LogisticRegression(), param_grid=params, cv=5)

# Обучаем модель с перебором параметров
grid.fit(X_train, y_train)

# Смотрим лучшие параметры и среднюю оценку по кросс-валидации
print(grid.best_params_)
print(grid.best_score_)
```

Объяснить код с 

`cv=5` означает, что каждая комбинация проверяется на пяти разбиениях. В итоге остаются параметры, которые стабильно показывают лучший результат.

В задачах с большим числом признаков или ресурсоёмкими моделями часто используют `RandomizedSearchCV`. Он не перебирает всё подряд, а выбирает случайные сочетания параметров из заданных диапазонов. Это экономит время на обучение без потери качества.

Настраиваем стратегию разбиения

Параметр `cv` управляет тем, как именно делятся обучающие данные при кросс-валидации. Как мы уже разобрали, в `scikit-learn` по умолчанию используется `KFold`, и для большинства задач он



Ты точно знаешь себя?

Проверь свой IT-профиль

Например, если у нас задача классификации и классы в выборке распределены неравномерно (что, кстати, бывает довольно часто), лучше использовать `StratifiedKFold`. Он сохраняет

пропорции классов в каждом разбиении. Это важно, чтобы избежать перекоса в обучении и не получить завышенную оценку.

Пример с StratifiedKFold :

```
from sklearn.model_selection import StratifiedKFold, cross_val_score
from sklearn.linear_model import LogisticRegression

# Стратегия: сохраняем пропорции классов в каждом разбиении
cv = StratifiedKFold(n_splits=5)

# Модель для классификации
model = LogisticRegression()

# Оцениваем модель на каждом фолде по метрике accuracy
scores = cross_val_score(model, X, y, cv=cv, scoring="accuracy")

# Смотрим оценки по фолдам и их среднее
print(scores)
print(f"Средняя точность: {scores.mean():.2f}")
```

Объяснить код с 

В этом примере используется пять разбиений (`n_splits=5`), а метрика — точность (`accuracy`). На выходе получится список значений, по одному на каждый фолд, и их среднее. Это даёт более надёжную оценку, чем один запуск, и помогает понять, как модель ведёт себя на разных частях данных.

Для задач с дополнительными требованиями, например с разбиениями по группам, временным интервалам или с повторениями, `scikit-learn` поддерживает и другие стратегии: `GroupKFold` , `TimeSeriesSplit` , `RepeatedKFold` . Но в большинстве случаев достаточно `KFold` или `StratifiedKFold` .

Сохраняем модель после настройки

Если модель уже обучена и проверена, каждый раз запускать `fit()` заново нет смысла, особенно если обучение шло долго или в процессе подбирались параметры. Гораздо проще сохранить итоговый объект.

В библиотеке `scikit-learn` для этого используют `joblib` . Он сохраняет модель вместе со всеми настройками и весами. После загрузки она ведёт себя так же — можно сразу вызывать `predict()` на новых данных.

```
import joblib
from sklearn.linear_model import LogisticRegression

# Обучаем модель
model = LogisticRegression()
model.fit(X_train, y_train)

# Сохраняем в файл
joblib.dump(model, "model.pkl")
```



Ты точно знаешь себя?

Проверь свой IT-профиль

```
# Предсказываем на новых данных
y_pred = model_loaded.predict(X_test)
```

[Объяснить код с](#) 

Так сохраняют не только финальную модель, но и промежуточные шаги, например обработку или пайплайн целиком. Это помогает избежать повторов, ускорить запуск и не терять уже отлаженные части.

[К содержанию](#)

Разбираем частые ошибки при работе с библиотекой

Даже если библиотека кажется простой, ошибки в коде всё равно случаются. Особенно вначале, когда шагов много, а часть обработки легко пропустить. Ниже — о том, с какими проблемами часто сталкиваются при работе с библиотекой `sklearn` и как их быстро решить.

1. Необработанные NaN или категориальные признаки.

Если в данных остались `NaN` или строки, `fit()` упадёт с ошибкой. `Scikit-learn` не работает с пропусками и текстовыми значениями напрямую — их нужно заранее обрабатывать.

Часто это происходит, если забыли про `SimpleImputer` или `OneHotEncoder`. Например, в пайплайне есть масштабирование, но нет заполнения пропусков. Или категориальный признак закодировали в `train`, но не в `test`.

Проверить можно через `df.info()`: он покажет пропуски и типы данных. Чтобы не допускать таких ошибок, лучше сразу собирать всю обработку в пайплайн.

В простых случаях пропуски можно заменить вручную: через `fillna()`, подставив значение, которое точно не встречается в столбце, например `-1`. Это удобно, если обработка выполняется до запуска пайплайна и не включает `SimpleImputer`.

2. Применение `fit_transform` до разбиения на выборки.

Одна из частых ошибок — сначала вызвать `fit_transform()` на всей таблице, а потом уже делить её на обучение и тест. В таком случае признаки на тесте уже будут учтены при обучении трансформера и модель получит доступ к данным, которые должна была видеть только после обучения.

Важно не допускать утечки информации между обучением и тестированием — иначе метрика будет завышенной и не покажет реальную точность модели.

Правильный порядок:

1. Разбить данные на обучающую и тестовую выборки.
2. Вызвать `fit()` — только на обучающей выборке.
3. Применить `transform` — отдельно к `train` и `test`.

Если шагов много, проще собрать их в пайплайн: он автоматически применяет `fit` и `transform` в нужной последовательности и не путает выборки.



Ты точно знаешь себя?

Проверь свой IT-профиль

некорректный результат. Частый пример: сначала применяют масштабирование, а потом пытаются заполнить пропуски. Но `StandardScaler` не работает с `NaN` и вызов `fit()` не проходит.

Правильнее сначала обрабатывать пропуски — через `SimpleImputer` — и только потом применять `StandardScaler` или другой трансформер. Порядок шагов важен: каждый следующий должен получать данные в том формате, который он умеет обрабатывать.

Путаница чаще всего возникает, если пайплайн собирается вручную и шаги не подписаны явно. В таких случаях помогает проверка на срезе данных или поочерёдный запуск `.fit_transform()` — чтобы убедиться, что каждый шаг работает с тем, что получил.

4. Путаница между `fit()`, `transform()` и `fit_transform()`.

В `scikit-learn` у трансформеров три метода, и на старте легко запутаться, когда вызывать каждый из них.

- `fit()` — обучает трансформер: считает, что и как менять (например, среднее и стандартное отклонение для масштабирования);
- `transform()` — применяет преобразование к новым данным;
- `fit_transform()` — объединяет оба шага: сначала обучение, потом применение.

На обучающей выборке обычно вызывают `fit_transform()`. А на тестовой — только `transform()`. Если случайно вызвать `fit()` на тесте, получится утечка информации.

В пайплайне всё это вызывается автоматически: `fit()` применяется только к обучающим данным, `transform()` — отдельно к обучающим и тестовым. Поэтому пайплайн почти полностью убирает эту путаницу.

5. Неправильный формат входных данных.

Для методов `fit()` и `predict()` нужны данные в определённой форме. Обычно это массивы (`ndarray`) или датафреймы с числовыми признаками. Если передать `Series`, `dict`, датафрейм с неправильными индексами или столбцами нестандартного типа, то модель может отработать некорректно или вообще не запуститься.

Ошибку можно получить сразу, например `ValueError` при несовпадении формы, но бывает, что модель обучается, а результат оказывается странным: метрика падает, значения повторяются или обрезаются.

Проверить проще всего через `X.shape`, `X.dtypes` и `X.head()`. Если данные подаются в `predict()` после обработки вручную — важно убедиться, что структура осталась той же, что и при `fit()`. Опять же, в пайплайне такие несостыковки случаются реже, но при отладке всё равно стоит это держать в голове.

6. Оценка модели на обучающих данных.

Частая ошибка — вызвать `score()` сразу после `fit()` и считать, что это и есть итоговое качество. Но если оценивать модель на тех же данных, на которых она обучалась, результат всегда будет завышенным, особенно если модель сложная.

Чтобы получить реалистичную оценку, нужно проверить результат на данных, которые модель не видела. Даже простого `train_test_split()` хватает, чтобы понять, переобучилась модель или нет.

Сейчас будет снова про пайплайн.



Ты точно знаешь себя?
Проверь свой IT-профиль

можно перейти к кросс-валидации, если нужно стабильнее.

7. Отсутствие заданного `random_state` .



Источник

Если не зафиксировать `random_state` , результат может меняться при каждом запуске. Данные разбиваются по-разному, а модели ведут себя чуть иначе — например, случайным образом выбирают начальные значения или подмножества для обучения. Из-за этого метрика может заметно меняться.

Особенно обидно, если вы сравниваете разные модели или логируете результаты экспериментов: кажется, что одна версия работает лучше, хотя на деле просто повезло.

Чтобы избежать таких ситуаций, стоит задавать `random_state` **везде, где он есть**. Он используется при разделении данных (`train_test_split`), в кросс-валидации (`KFold` , `StratifiedKFold`), в подборе параметров и в моделях с внутренней случайностью, например: `RandomForest` , `LogisticRegression` , `KMeans` . Это простое правило помогает сделать результат стабильным и воспроизводимым.

8. Несовместимость версий библиотек.

Код может быть написан правильно, но всё равно не работать или вести себя странно. Часто дело в несовместимости версий: где-то аргумент устарел, где-то поменялась структура данных. Такое бывает между разными версиями `scikit-learn` , `pandas` и `numpy`.

Если результат внезапно стал другим: появились предупреждения, ошибка в неожиданном месте или просто «поехало поведение», — стоит:

- проверить версии через `pip list` или `conda list` ;
- свериться с документацией нужной версии;
- использовать виртуальные среды (`venv` , `conda` , `poetry`) и фиксировать зависимости в `requirements.txt` или `pyproject.toml` .

Проблемы чаще всего всплывают при переносе проекта: с ноутбука на сервер, между коллегами или просто после долгого перерыва.



Ты точно знаешь себя?

Проверь свой IT-профиль

Если метрика внезапно близка к идеальной, это почти всегда сигнал, что что-то пошло не так.

Частые причины:

- признаки из теста оказались в обучении — утечка;
- в таблице оказался признак, напрямую связанный с целевой переменной — например, метка класса под другим именем;
- `fit_transform()` вызван до разбиения и тестовые данные уже учтены при обучении трансформера.

В этих случаях `accuracy` или `roc_auc` могут быть близки к 1, но на новых данных модель быстро теряет качество.

Что проверить:

- какой порядок шагов: когда были разделены данные, где вызывался `fit()`, применялись ли трансформеры ко всей таблице;
- что именно входит в набор признаков — нет ли в нём лейбла или его производных;
- как ведёт себя модель при отложенной валидации или на кросс-валидации.

Такие ошибки не всегда видны сразу, но их проще отследить, если собирать пайплайн и чётко разделять данные на этапах. В остальном — проверка логики важнее любой метрики.

К содержанию

Что изменилось в scikit-learn в 2025

Библиотека убрала часть рутинных действий, упростила отладку и сделала поведение более предсказуемым. Ниже — по пунктам, что поменялось и как это можно использовать на практике.

Мы уже учли их в гайде, а здесь собираем воедино.

1. Полная поддержка `pandas` в трансформерах.

Раньше, даже если подавался `DataFrame`, трансформеры `scikit-learn` почти всегда возвращали `numpy`. Названия колонок терялись, и на выходе получался массив без структуры. Это усложняло отладку: нужно было вручную оборачивать результат в `DataFrame`, подставлять имена признаков, следить за порядком.

Теперь, если на вход подать таблицу, `transform()` вернёт таблицу — с нужными индексами, названиями колонок и правильными типами. Это касается всех стандартных трансформеров: `OneHotEncoder`, `StandardScaler`, `SimpleImputer` и других.

Что это даёт:

- можно в любой момент сделать `X_transformed.head()` и сразу увидеть результат обработки;
- удобно сохранять промежуточные данные, если нужно понять, как ведёт себя модель;
- меньше риска перепутать признаки при ручной сборке пайплайна или логировании.



Ты точно знаешь себя?

Проверь свой IT-профиль

```
from sklearn.preprocessing import OneHotEncoder
import pandas as pd
```

```
df = pd.DataFrame({"region": ["North", "South", "East", "North"]})
df["region"] = df["region"].astype("category")

encoder = OneHotEncoder(sparse_output=False)
encoded = encoder.fit_transform(df)

# Раньше: encoded — это массив без имён колонок
# Сейчас: если df подать в ColumnTransformer, на выходе сохранится DataFrame

from sklearn.compose import ColumnTransformer

ct = ColumnTransformer([
    ("region", OneHotEncoder(sparse_output=False), ["region"])
], remainder="passthrough")

result = ct.fit_transform(df)

# Результат — DataFrame с названиями колонок, например:
# region_East, region_North, region_South
```

Объяснить код с 🦊

2. Обновления в Pipeline и ColumnTransformer .

В новых версиях scikit-learn упростили работу с пайплайнами. Главное — ошибки теперь легче отслеживать. Если на каком-то шаге что-то пошло не так, сообщение об ошибке укажет конкретный блок, а не просто выдаст длинный traceback.

В ColumnTransformer теперь можно выбирать колонки автоматически — с помощью `make_column_selector()` . Он определяет нужные признаки по типу данных. Например, все числовые — в масштабирование, все категориальные — в кодировку.

Когда это полезно:

- если таблица приходит из внешнего источника и в ней периодически меняется состав признаков;
- если таблица собирается динамически, например после отбора фич;
- если пайплайн используется повторно — на новых данных или в другой задаче.

Пример:

```
from sklearn.compose import ColumnTransformer, make_column_selector
from sklearn.preprocessing import StandardScaler, OneHotEncoder

preprocessor = ColumnTransformer([
    ("num", StandardScaler(), make_column_selector(dtype_include="number")),
    ("cat", OneHotEncoder(), make_column_selector(dtype_include="category")),
])
```

Объяснить код с 🦊



Ты точно знаешь себя?

Проверь свой IT-профиль

автоматически попадут в нужную обработку.

3. Новая реализация бустинга.

Раньше в scikit-learn был градиентный бустинг (GradientBoosting*), но он строил деревья на основе точных значений признаков и плохо масштабировался.

В новых версиях основной стала реализация на гистограммах —

HistGradientBoostingClassifier и HistGradientBoostingRegressor . Она сначала разбивает признаки на интервалы (бины), а затем строит деревья по этим диапазонам. Такой подход ускоряет обучение, снижает потребление памяти и стабильно работает даже при глубокой структуре и большом числе признаков.

Модель умеет:

- работать с пропусками — это исключение среди моделей scikit-learn: другие требуют предварительного заполнения;
- обрабатывать категориальные признаки напрямую, если задать тип category ;
- использовать early stopping — остановку обучения, — если качество на валидации перестает расти;
- рассчитывать важность признаков через .feature_importances_ или permutation_importance() .

Что это даёт:

- сильную и быструю модель — без внешних зависимостей;
- стабильную работу на больших датасетах;
- встроенные средства анализа — можно сразу понять, какие признаки влияют на результат.

Пример:

```
from sklearn.ensemble import HistGradientBoostingClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

model = HistGradientBoostingClassifier()
model.fit(X_train, y_train)
y_pred = model.predict(X_test)

print(f"Accuracy: {accuracy_score(y_test, y_pred):.2f}")
```

Объяснить код с 

Функция train_test_split делает простой split данных: делит выборку на обучающую и тестовую. Отдельная предобработка не требуется, если признаки уже числовые или категориальные.

4. Фиксация и контроль параметров.



Ты точно знаешь себя?

Проверь свой IT-профиль

применяют её уже с другими. В пайплайне такие случаи особенно трудно отследить.

Теперь поведение стало строже. Если модель уже обучена и вы меняете важный параметр, например `max_depth` у дерева или `C` у логистической регрессии, `scikit-learn` выдаст предупреждение или ошибку. Это защита от случайных изменений, которые могут повлиять на результат. Разработчику не нужно отслеживать вручную, сбросились ли веса: всё контролируется автоматически.

Часто параметры меняют через метод `set_params()`. Его применяют в `GridSearchCV` или вручную, если нужно перебрать конфигурации. Раньше при таком вызове могла остаться обученная часть: веса, коэффициенты, структура дерева. Сейчас при любом изменении параметров объект сбрасывает состояние и требует повторного обучения. Это упрощает отладку и делает поведение моделей более предсказуемым в разработке и тестировании.

Пример кода с `set_params()` :

```
from sklearn.linear_model import LogisticRegression

model = LogisticRegression(C=1.0)
model.fit(X_train, y_train)

# Меняем параметр после обучения
model.set_params(C=0.1)

# Пытаемся сделать предсказание без повторного fit
model.predict(X_test)
```

Объяснить код с 🐞

Результат:

```
NotFittedError: This LogisticRegression instance is not fitted yet.
Call 'fit' with appropriate arguments before using this estimator.
```

Объяснить код с 🐞

5. Мелкие улучшения, которые экономят время.

Никаких новых алгоритмов, но стало удобнее работать с привычными вещами.

Можно задавать параметры прямо в пайплайне

Раньше, чтобы настроить шаг внутри пайплайна, нужно было указывать название шага и параметр:

```
pipe.set_params(preprocessor__num__with_mean=False)
```

Объяснить код с 🐞

Это сокращается, но теперь параметры можно задавать и при создании пайплайна:



Ты точно знаешь себя?

Проверь свой IT-профиль

```
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
```

```
pipe = make_pipeline(StandardScaler(with_mean=False), LogisticRegression(C=0.5))
```

[Объяснить код с](#) 

Такой способ проще, если не нужен `GridSearchCV`, а параметры известны заранее.

Появились короткие вызовы для трансформеров

Некоторые классы теперь поддерживают сокращённые вызовы — вместо `transform()` можно сразу использовать объект как функцию:

```
scaler = StandardScaler()
X_scaled = scaler.fit(X).transform(X)
# Теперь можно так:
X_scaled = scaler.fit(X)(X)
```

[Объяснить код с](#) 

Это не критично, но сокращает количество вызовов в цепочках и визуально упрощает код.

Сообщения об ошибках стали понятнее

Если что-то не так с размерностью, типами данных или параметрами, `scikit-learn` теперь чаще указывает, на каком именно этапе возникла ошибка — особенно внутри пайплайнов и `ColumnTransformer`.

[К содержанию](#)

Чему учиться после sklearn

Работа с текстами и изображениями

`Scikit-learn` работает только с табличными данными. Если нужно анализировать текст, изображения или звук — подключается другой стек.

Для текстов стоит изучить:

- `spaCy` — для базовой обработки и извлечения признаков;
- `transformers` от Hugging Face — для работы с современными языковыми моделями.

Если нужно работать с изображениями — например, классифицировать, сегментировать или генерировать, — дальше идёт `PyTorch`, `TensorFlow` и `Keras`. Эти библиотеки используют нейросети и часто применяются в компьютерном зрении: вместо таблиц они работают с изображениями, последовательностями и тензорами.

Мощные модели ML для табличных данных



Ты точно знаешь себя?

Проверь свой IT-профиль

`scikit-learn`, поэтому легко встроить их в текущий пайплайн.

Хорошая точка старта — документация CatBoost, гайд по LightGBM и официальные примеры XGBoost.

Автоматизация подбора параметров и экспериментов

Когда модель становится сложнее, а вариантов конфигурации всё больше, стоит перейти от ручного подбора к автоматическому.

Если используете scikit-learn — начните с GridSearchCV и RandomizedSearchCV. Дальше можно изучить Optuna — библиотеку для интеллектуального подбора параметров с логированием, визуализациями и поддержкой пайплайнов.

Чтобы не терять настройки, стоит добавить логирование экспериментов через MLflow или Weights & Biases.

Управление данными и зависимостями

Если в проекте несколько моделей, обновляются датасеты и приходится возвращаться к старым запускам — имеет смысл использовать Data Version Control (DVC).

Это инструмент, который отслеживает версии данных и моделей, работает вместе с Git и помогает воспроизвести результат в любой момент.

Посмотрите официальный гайд по DVC: он покажет, как фиксировать данные, конфигурации и пайплайны.

Разбор внутренней логики моделей машинного обучения

Scikit-learn закрывает математику за простым API, но при нестабильном поведении модели важно понимать, что происходит внутри.



Источник

Имеет смысл углубиться в такие темы, как регуляризация, градиентный спуск, переобучение, вероятностные модели. Это поможет не только исправлять ошибки, но и строить более устойчивые



Ты точно знаешь себя?

Проверь свой IT-профиль

Выход в продакшн

Если модель стабильно работает и даёт результат, её можно подключить к реальной системе. Сначала сохранить: вручную — через `joblib.dump(model, 'model.pkl')` — или с помощью **MLflow** — он дополнительно логирует параметры, метрики и версии, что удобно при работе с несколькими экспериментами.

Потом — развернуть сервис через FastAPI, который примет входные данные, передаст их модели и вернёт результат. Добавить логирование, простую валидацию и пару тестов — и получится полноценный микросервис, который можно подключать к продукту.

Подытожим

Scikit-learn Python — это библиотека, с которой начинается большинство реальных задач. Она про то, чтобы быстро собрать прототип, проверить гипотезу и не закопаться в инфраструктуре. Здесь видно, что именно делает ML-модель, и понятно, где править, если что-то пошло не так.

Когда выстроена база — с пайплайнами, метриками, нормальной отладкой, — дальше проще пробовать новые библиотеки и фреймворки.



Шутки шутками, но в какой бы стек вы ни ушли, привычка собирать задачу аккуратно, как в scikit-learn, остаётся с вами.

Стать аналитиком, который умеет превращать данные в понятные инсайты, и открыть большие перспективы в карьере позволяет программа повышения квалификации «Python для анализа данных» от Нетологии. Мы обучаем по государственной лицензии и выдаём удостоверение установленного образца. Наши студенты достигают своих целей: их реальные истории можно почитать здесь.

Теги: install, перепессия, sklearn, scikit-learn, datasets, python машинное обучение, python sklearn, машинное обучение в python, модели классификации sklearn, python анализ данных

Хабы: Блог компании Нетология. Машинное обучение. Python. Data Engineering. Data Mining



Ты точно знаешь себя?
Проверь свой IT-профиль

Редакторский дайджест

Присылаем лучшие статьи раз в месяц

Подписаться

Оставляя почту, я принимаю Политику конфиденциальности и даю согласие на получение рассылок



Нетология

Знания, на которые можно положиться

Сайт



13

19

Карма Общий рейтинг

@kirakirap

IT-редактор и очень любознательная персона

Подписаться



 Комментарии 3

Публикации

ЛУЧШИЕ ЗА СУТКИ ПОХОЖИЕ

- 

stas_dubich

17 часов назад
- IT-2025: Реквием по здравому смыслу
-  Простой

 12 мин

 41K
- Мнение
-  +143

 115

 117



Loxmatiyamamont

21 час назад

История поиска бага в ядре Linux длиной в год, или неожиданные нули из XFS'a

 Сложный

 11 мин

 9.9K

 +46

 19

 6



ultra_vds

23 часа назад

FTP-сервер на Linux: настройка для домашнего проекта

 Средний

 11 мин

 9.3K

Тutorial

 +38

 51

 9



Ты точно знаешь себя?
Проверь свой IT-профиль

Путь Солнца Золотого

 Простой  21 мин  7.6K

+30

16

12



Lunathecat

20 часов назад

Электрогитара с семьёю «вкусными» звучаниями, доступными моментально

Простой

9 мин

10K

[Тutorial](#)

+30

9

0



PCB-network

20 часов назад

20 лет в индустрии: история команды SimPCB

9 мин

8K

[Интервью](#)

+28

7

12



KarinaKosheva

23 часа назад

Теория и практика интеграции СХД с OpenStack

Средний

13 мин

7.1K

[Tutorial](#)

+27

11

2



proDream

22 часа назад

GitLab: Основы написания Pipeline 1/3

Простой

15 мин

9.4K

[Tutorial](#)

+24

152

6



cognitronn

20 часов назад

Почему индустрия ИИ начинает трещать по швам?

4 мин

8.8K

[Мнение](#)[Перевод](#)

+19

14

51



Xcom-shop

23 часа назад

Как раньше выбирали компьютеры: ностальгический чек-лист из 2000-х



Ты точно знаешь себя?

Проверь свой IT-профиль

Упрощаем клиентский путь с помощью Цифрового профиля

Турбо

Показать еще

МИНУТОЧКУ ВНИМАНИЯ



Событие

Онлайн и офлайн: зимние IT-ивенты в Календаре



IT-комикс

Инцидент на трубопроводе: расследование в Heavy Digital



Турбо

Клиринг работает, просто не суетится

КУРСЫ

 1С-программист

По мере набора группы

 Профессия Графический дизайнер PRO

По мере набора группы

 KL 004.2.4 Kaspersky SD-WAN

28 января

 Профессия Product Manager

По мере набора группы

 Python-разработчик

По мере набора группы

Больше курсов на Хабр Карьере

ИНФОРМАЦИЯ

Сайт	netology.ru
Дата регистрации	10 марта 2017
Дата основания	2011
Численность	501–1 000 человек
Местоположение	Россия
Представитель	Мария Верховцева

ВИДЖЕТ



Ты точно знаешь себя?

Проверь свой IT-профиль



БЛОГ НА ХАБРЕ

23 дек в 14:31

Классы в Python: от основ ООП до продвинутых концепций

10K 10

9 дек в 14:12

Системный инженер или разработчик: что выбрать в 2026 году

11K 2

20 ноя в 19:02

Docker для начинающих: что это такое и как пользоваться

33K 11

29 окт в 15:58

Вся правда о карьере в ИБ: разбираем мифы про хакеров и высокие зарплаты

17K 9

14 окт в 17:06

Android для новичков: как не наломать дров с корутинами и какие привычки оставить в прошлом

8.4K 7

ИСТОРИИ

Настоящее и будущее data: исследование

X5 Tech и Хабр опросили более 500 data-специалистов, чтобы больше узнать об их работе, проблемах и трендах в data-сфере. Вот чем они поделились.

Код без границ: кто получил гранты?

AI-инновации Наука и образование без границ

Для кого и как? Разработка для разработчиков: инструменты и библиотекы

да, это новый ITaaS/РaaS-облако

запустили

Хабр Карьера

Выбирайте обучение по рейтингам и отзывам

Ты точно знаешь себя? Проверь свой IT-профиль

Как меняется работа в data

Код без границ: кто получил гранты?

Год облака в подарок: 2 VM, БД, 100 ГБ

IT-анатомия идеального ремонта

«Зелёная» подборка

Выбирайте курсы по отзывам

На мороз!

Ваш аккаунт

Войти

Регистрация

Разделы

Статьи

Новости

Хабы

Компании

Авторы

Песочница

Информация

Устройство сайта

Для авторов

Для компаний

Документы

Соглашение

Конфиденциальность

Услуги

Корпоративный блог

Медийная реклама

Нативные проекты

Образовательные программы

Стартапам







Настройка языка

Техническая поддержка

© 2006–2025, Habr



Ты точно знаешь себя?

Проверь свой IT-профиль